



Chunking, Metadata & Embedding Pipeline Implementation



Syarat dan Aturan dalam Pelatihan

Untuk mengikuti materi pelatihan berikut ada beberapa hal yang harus diperhatikan oleh peserta:

- Peserta diharapkan sudah memiliki pengetahuan dasar tentang Python, Dasar Pengolahan Data, Embedding, dan Vector Database.
- Peserta diharapkan memperhatikan materi dan mencatat materi atau penjelasan yang disampaikan dari instruktur.
- Instruktur diharapkan dapat menjelaskan materi dengan jelas dan memastikan peserta pelatihan dapat memahami topik yang dibahas.
- Materi yang diajarkan sebagian besar adalah praktik atau menggunakan Google Colab (Jupyter Notebook).



Daftar Isi

- Embedding Pipeline Design
 - Data Ingestion
 - Chunking Strategies
 - Metadata
 - Model Embedding
- Hybrid Search & Pre-RAG
- Common Failure Patterns
- Hands on: Store embeddings in vector DB, retrieval quality test



Objectives

- Peserta dapat menjelaskan dan memahami proses dalam melakukan Embedding, Chunking, dan Membuat Metadata.
- Peserta dapat memahami langkah-langkah untuk melakukan Hybrid Search.
- Peserta mampu mempraktikkan atau mensimulasikan proses Embedding dan Hybrid Search.



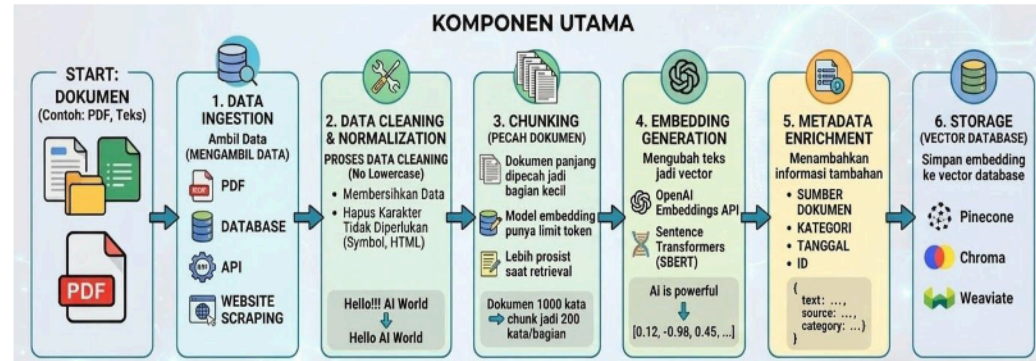
Embedding Pipeline Design

Embedding Pipeline

Proses Embedding

Embedding pada dasarnya adalah proses untuk mengubah data ke dalam bentuk vektor agar dapat di proses oleh model AI sebelum melakukan pencarian data atau di simpan ke Vector Database.

Dalam Proses Embedding Ada beberapa komponen utama yang harus diperhatikan agar menghasilkan respon yang bagus atau akurat, diantaranya yaitu Jenis Dokumen dan Cara Membacanya, Data Cleaning, Chunking, dan Model Embedding.



Embedding Pipeline

Data Ingestion

Secara umum, RAG bisa pakai hampir semua jenis data seperti PDF, Word (DOCX), TXT, Markdown, Website / HTML, Database, Spreadsheet, dll.

Kalau sumbernya campur:

- Format beda-beda
- Struktur beda
- Kualitas beda

Ini akan membuat pipeline jadi lebih kompleks. Solusinya kita dapat menggabungkan semua informasi tersebut ke dalam 1 dokumen (PDF) atau memberikan metadata untuk memberitahu sumber dokumennya.

Metadata

```
{  
  "text": "AI digunakan untuk  
prediksi...",  
  "source": "pdf",  
  "document name": "modul ai.pdf",  
  "category": "AI",  
  "timestamp": "2025-01-01"  
}
```

Embedding Pipeline

Data Ingestion

Berikut ini adalah library yang dapat digunakan untuk membaca file PDF:

Library	Kelebihan	Kekurangan	Cocok untuk
pypdf	Ringan, simple, mudah digunakan	Layout sering berantakan, kurang bagus untuk tabel/kolom	✓ PDF teks sederhana ✓ Preprocessing cepat ✓ Pipeline ringan
PyMuPDF F	Lebih akurat, support layout kompleks, bisa ekstrak tabel	Sedikit lebih kompleks dari pypdf	✓ PDF dengan tabel ✓ Dokumen multi-kolom ✓ Analisis dokumen bisnis
pymupdf 4llm	Output Markdown rapi, siap untuk LLM/RAG	Keterbatasan pada Tata Letak Kompleks (struktur tidak beraturan atau tumpang tindih)	✓ RAG pipeline ✓ LLM ingestion ✓ Knowledge base AI

Chunking

Apa itu Chunking?

Chunking adalah teknik untuk memecah dokumen besar menjadi bagian-bagian kecil (chunks) sebelum dibuat embedding atau dipakai di RAG.

Tujuan dari chunking adalah supaya model lebih paham konteks, retrieval lebih akurat, dan tidak kena limit token.

Aspek	Tanpa Chunking	Dengan Chunking
Ukuran Input	Dokumen besar langsung diproses	Dipecah jadi bagian kecil
Kesesuaian Token Limit	✗ Sering melebihi limit model	✓ Aman (terkontrol)
Kualitas Embedding	✗ Campur banyak topik (blur)	✓ Lebih fokus per topik
Granularitas Informasi	✗ Terlalu umum	✓ Detail & spesifik
Konteks yang Diambil	✗ Bisa terlalu banyak / tidak relevan	✓ Lebih terarah
Efisiensi Query	✗ Berat (dokumen besar)	✓ Lebih ringan

Chunking

Chunking Strategies

Strategy	Cara Kerja	Kelebihan	Kekurangan
Fixed Chunking	Memecah teks berdasarkan jumlah karakter/token tetap tanpa memperhatikan struktur atau makna	✓ Sederhana dan mudah diimplementasikan ✓ Cepat diproses	✗ Bisa memotong kalimat di tengah ✗ Konteks bisa hilang ✗ Kurang akurat untuk retrieval
Chunking + Overlap	Memecah teks seperti fixed chunking, tetapi setiap chunk memiliki bagian yang saling tumpang tindih	✓ Menjaga kontinuitas konteks antar chunk ✓ Retrieval lebih akurat dibanding fixed ✓ Cocok untuk RAG	✗ Data menjadi lebih banyak (redundant) ✗ Membutuhkan storage lebih besar
Recursive Chunking	memecah teks secara bertahap berdasarkan paragraf, kalimat atau kata. Bisa juga berdasarkan thematic section.	✓ Sangat relevan secara konteks ✓ Tidak memotong informasi penting ✓ Akurasi retrieval tinggi	✗ Implementasi lebih kompleks ✗ Lebih lambat dalam pemrosesan dibanding metode lain

Chunking

Fixed Chunking

Misalkan kita memiliki input seperti berikut:

Artificial Intelligence is transforming the world. It helps automate tasks and improve efficiency in many industries. AI is widely used in healthcare and finance.

Kemudian kita mencoba memecahnya dengan **chunk_size = 50 karakter**, tanpa memperhatikan kalimat atau makna.

Output

Chunk 1:

Artificial Intelligence is transforming the world.

Chunk 2:

It helps automate tasks and improve efficiency in

Chunk 3:

many industries. AI is widely used in healthcare

Chunk 4:

and finance.

Chunking

Fixed Chunking

Kita bisa juga menambahkan **separator** sebagai pemisah antara chunk, misalkan ingin dipisahkan berdasarkan titik. Berikut adalah contoh kode dengan LangChain.

```
splitter = CharacterTextSplitter(  
    chunk_size=50,  
    chunk_overlap=0,  
    separator=".",  
)
```

Output

Chunk 1:

Artificial Intelligence is transforming the world

Chunk 2:

It helps automate tasks and improve efficiency in many industries

Chunk 3:

AI is widely used in healthcare and finance

Chunking

Fixed Chunking + Overlap

Kita bisa juga menambahkan **chunk_overlap** sebagai untuk mengatur bagian teks yang sengaja diulang (ditumpang tindihkan) di antara dua chunk yang berurutan.

Pada contoh berikut agar overlapnya keliatan kita atur separatornya jadi satu spasi.

```
splitter = CharacterTextSplitter(  
    chunk_size=50,  
    chunk_overlap=2,  
    separator=" ",  
)
```

Output

Chunk 1:

Artificial Intelligence is transforming the world.

Chunk 2:

It helps automate tasks and improve efficiency in

Chunk 3:

in many industries. AI is widely used in

Chunk 4:

in healthcare and finance.

Chunking

Recursive Chunking

Pada bagian ini, misalkan kita ingin melakukan Recursive Chunking dengan urutan pemecahan:

1. Coba split berdasarkan paragraf (

)
2. Kalau terlalu panjang → split kalimat (.)
3. Kalau masih panjang → split kata ()

Untuk mengatur urutan pemecahan tersebut kita bisa atur dengan **separator**.

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=120,  
    chunk_overlap=20,  
    separators=["  
  
", ". ", " "],  
    keep_separator=False  
)
```

Chunking

Recursive Chunking

Misalkan kita punya teks seperti di bawah ini, yang terdiri dari 3 paragraf. Karena paragraf 1 dan 2 terlalu panjang jika kita menggunakan kode sebelumnya dengan pengaturan `chunk_size` dan separator yang sama, maka 1 paragraf akan dipisah menjadi beberapa chunk karena terlalu panjang atau lebih dari 120 karakter.

```
Artificial Intelligence is transforming many industries and changing  
how businesses operate across the world. It enables automation and  
improves efficiency in various sectors.
```

```
In healthcare, AI is widely used to help doctors diagnose diseases more  
accurately and efficiently. AI models can process large datasets  
quickly and provide insights that were previously difficult to obtain.
```

```
In finance, AI is applied for fraud detection and risk analysis. Many  
banks rely on machine learning to improve decision-making.
```

Chunking

Recursive Chunking

Output dari data sebelumnya adalah menghasilkan 6 potongan (chunk) :

Chunk 1:

Artificial Intelligence is transforming many industries and changing how businesses operate across the world

Chunk 2:

It enables automation and improves efficiency in various sectors.

Chunk 3:

In healthcare, AI is widely used to help doctors diagnose diseases more accurately and efficiently

Chunking

Recursive Chunking

Chunk 4:

AI models can process large datasets quickly and provide insights that were previously difficult to obtain.

Chunk 5:

In finance, AI is applied for fraud detection and risk analysis

Chunk 6:

Many banks rely on machine learning to improve decision-making.

Sekarang cobalah ubah `chunk_size` menjadi 200, berapa total chunk yang diperoleh?

Metadata

Apa itu Metadata?

Metadata adalah suatu data atau informasi tambahan yang dapat membantu untuk menjelaskan, mengelompokkan, atau memberi konteks pada suatu dokumen.

Dalam beberapa kasus, misalkan ketika menggunakan Pinecone, isi konten atau teks asli dapat dimasukkan ke metadata, karena Pinecone tidak memiliki field khusus untuk dokumen atau konten. Hal ini dilakukan karena kita butuh teks asli untuk dikirim ke LLM.

Kegunaan Metadata

- Membuat Search Lebih Presisi karena bisa filter dokumen atau data.
- Mengurangi noise pada hasil pencarian dengan mengeliminasi dokumen yang tidak relevan atau membatasi domain pencarian.
- Membantu dalam segmentasi data.

Metadata

Struktur Penyimpanan Data

Dalam Proses Penyimpanan Metadata dan Embedding terutama ketika menggunakan ChromaDB dan Pinecone, keduanya memiliki perbedaan. Di ChromaDB teks konten atau dokumen disimpan dalam field **documents** dan terpisah dengan metadata, sedangkan di Pinecone teks konten, dokumen atau informasi lain disimpan dalam **metadata**.

Di ChromaDB nilai embedding (vektor) disimpan ke field **embeddings** sedangkan di Pinecone disimpan dalam **values**.

ChromaDB

```
collection.add(  
  documents=["Isi teks"],  
  embeddings=embeddings,  
  metadatas=[{"brand": "Nike"}],  
  ids=["1"]  
)
```

Pinecone

```
index.upsert(  
  {  
    "id": "1",  
    "values": embeddings,  
    "metadata": {  
      "text": "Isi teks",  
      "brand": "Nike"  
    }  
  }  
)
```

Metadata

Metadata Untuk Filtering

Misalkan kita punya data seperti berikut:

```
[{'id': 1,
  'kategori': 'sepatu',
  'brand': 'Nike',
  'harga': 950000,
  'rating': 4.7,
  'stok': True},
 {'id': 2,
  'kategori': 'sepatu',
  'brand': 'Adidas',
  'harga': 850000,
  'rating': 4.5,
  'stok': True},
```

Dari Metadata tersebut kita coba ambil **top 3** sepatu yang **harganya dibawah 900000** dan **brandnya Nike**.

ChromaDB

```
collection.query(
  query_texts=["sepatu"],
  n_results=3,
  where={
    "brand": "Nike",
    "harga": {"$lt": 900000}
  }
)
```

Pinecone

```
index.query(
  vector=query_embedding,
  filter={
    "brand": {"$eq": "Nike"},
    "harga": {"$lt": 900000}
  },
  include_metadata=True
)
```

Metadata

Operator dalam Filtering

Operator Perbandingan	Nama Lengkap	Artinya	Contoh
\$eq	Equal	Sama dengan (=)	<code>brand = "Nike"</code>
\$ne	Not Equal	Tidak sama (!=)	<code>brand != "Nike"</code>
\$gt	Greater Than	Lebih besar (>)	<code>harga > 500000</code>
\$gte	Greater Than or Equal	Lebih besar sama dengan (>=)	<code>rating >= 4.5</code>
\$lt	Less Than	Lebih kecil (<)	<code>harga < 900000</code>
\$lte	Less Than or Equal	Lebih kecil sama dengan (<=)	<code>Harga <= 1000000</code>

Metadata

Operator dalam Filtering

Operator	Nama Lengkap	Artinya	Contoh
\$and	Logical AND	Semua kondisi harus terpenuhi	<code>brand = Nike AND harga < 900000</code>
\$or	Logical OR	Salah satu kondisi terpenuhi	<code>brand = Nike OR brand = Adidas</code>

Metadata

Operator dalam Filtering

Operator Perbandingan	Nama Lengkap	Artinya	Contoh
\$eq	Equal	Sama dengan (=)	<code>brand = "Nike"</code>
\$ne	Not Equal	Tidak sama (!=)	<code>brand != "Nike"</code>
\$gt	Greater Than	Lebih besar (>)	<code>harga > 500000</code>
\$gte	Greater Than or Equal	Lebih besar sama dengan (>=)	<code>rating >= 4.5</code>
\$lt	Less Than	Lebih kecil (<)	<code>harga < 900000</code>
\$lte	Less Than or Equal	Lebih kecil sama dengan (<=)	<code>stok <= 10</code>

Embedding

Model Embedding

Model embedding adalah model AI yang mengubah data (teks, gambar, dll) menjadi vector (angka) sehingga bisa dipahami oleh mesin.

Contoh:

"sepatu lari" → [0.12, -0.45, 0.89, ...]

Vector ini merepresentasikan makna (semantic meaning), bukan sekadar kata.

Kegunaan Model Embedding

- Semantic Search: pencarian data berdasarkan makna.
- Retrieval dalam RAG : mencari dokumen yang relevan.
- Clustering & Similarity: mengelompokkan data yang mirip.
- Recommendation System: menyarankan item yang mirip berdasarkan teks atau gambar.

Embedding

Model Embedding

Provider	Model	Tipe	Keterangan
Sentence Transformers	all-MiniLM-L6-v2	Teks	Cepat, ringan, populer untuk semantic search
	all-mpnet-base-v2	Teks	Lebih akurat, dimensi lebih besar
	multi-qa-MiniLM-L6-cos-v1	Teks	Optimized untuk QA / retrieval
	clip-ViT-B-32	Image + Teks	Multimodal (CLIP) untuk image-text similarity

Embedding

Model Embedding

Provider	Model	Tipe	Keterangan
OpenAI	text-embedding-3-small	Teks	Cepat & murah (default use case)
	text-embedding-3-large	Teks	Akurasi tinggi (RAG production)
	CLIP (Terpisah dari OpenAI)	Image + Teks	Untuk similarity image-text

Embedding

Model Embedding

Provider	Model	Tipe	Keterangan
Google DeepMind (Gemini)	embedding-001	Teks	Embedding model resmi Gemini yang sering digunakan untuk teks.
	Gemini multimodal embedding	Image + Teks	Untuk cross-modal search



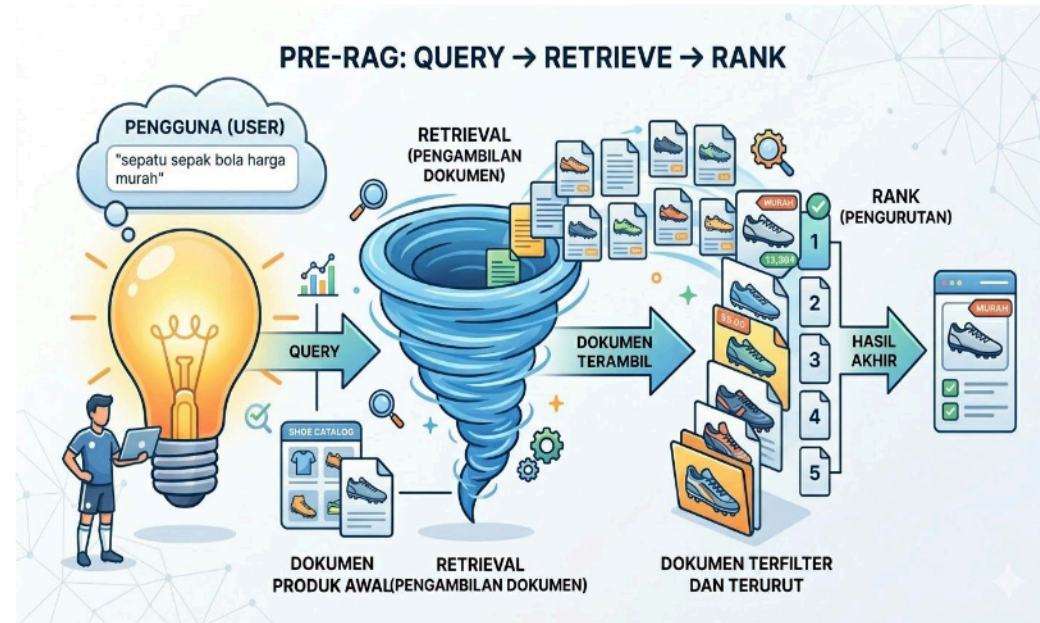
Pre-RAG & Hybrid Search

Pre-RAG

Definisi Pre-RAG

Pre-RAG (Pre-Retrieval Augmented Generation) adalah semua proses yang dilakukan sebelum model AI (LLM) menghasilkan jawaban.

Pada materi sebelumnya sebenarnya kalian telah mempelajari bagian dari Pre-RAG. Pada bagian ini kita akan coba bagi menjadi 3 komponen dasar dari Pre-RAG yaitu Query, Retrieve dan Rank.



Ilustrasi Pre-RAG

Pre-RAG

Query

Query adalah input dari user yang ingin dicari jawabannya oleh sistem RAG.

Contoh:

"sepatu lari murah untuk marathon"

Semakin jelas atau detail query yang diberikan user maka outputnya juga akan semakin tepat.

Proses Query

Proses yang bisa dilakukan dari Query adalah:

- Memasukan input teks atau data dari user.
- Cleaning / Normalization (opsional) : Misalkan "Sepatu Lari Murah!!!" → "sepatu lari murah"
- Keyword Extraction: Digunakan untuk BM25 (keyword search) atau filtering.
- Embedding (Query diubah jadi vector): "sepatu lari murah" → [0.21, -0.33, 0.87, ...]

Pre-RAG

Retrieve dan Rank

Retrieve adalah proses mengambil dokumen/data yang relevan dari database berdasarkan query user.

Hasil dokumen tersebut akan diurutkan atau re-ranking berdasarkan kemiripannya misalkan dengan cosine similarity.

Proses Retrieve dan Rank

Proses yang dilakukan adalah:

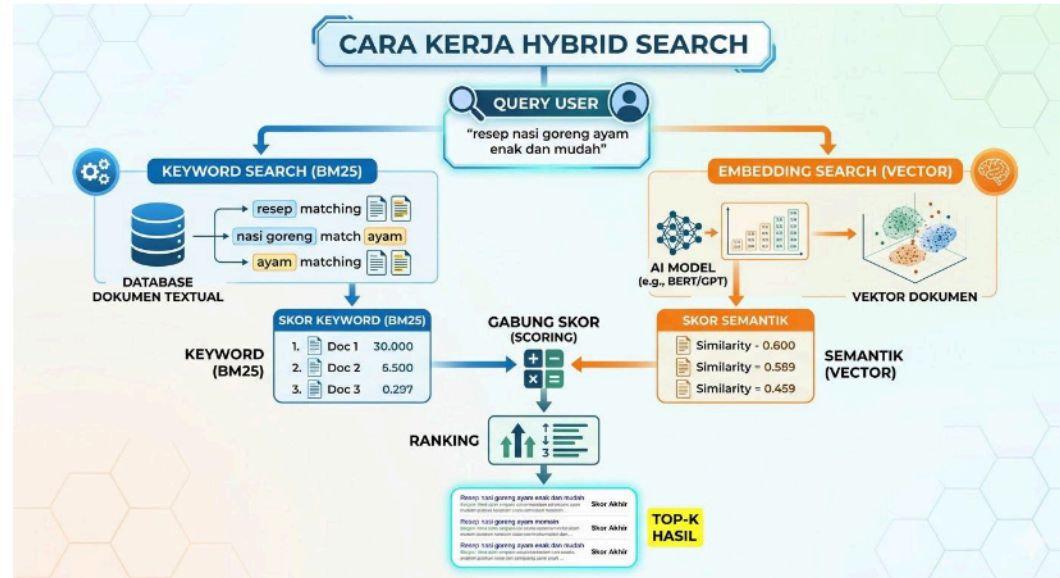
- Membandingkan kemiripan antara query dan dokumen berdasarkan keyword atau nilai embedding yang telah dihitung dengan matriks seperti cosine similarity.
- Melakukan filter berdasarkan kategori tertentu.
- Mengambil Top-K kandidat terbaik dari kumpulan data.
- Mengurutkan data berdasarkan dokumen yang paling mirip dengan query (input user).

Hybrid Search

Definisi Hybrid Search

Hybrid Search adalah teknik pencarian yang menggabungkan Keyword Search (Sparse) dan Semantic Search (Dense / Embedding).

Tujuan dari Hybrid Search adalah Mendapatkan hasil yang lebih **akurat, relevan, dan tidak miss informasi penting**.



Hybrid Search

Menggabung Scoring

Untuk Penggabungan skor kita dapat menggunakan **Weighted Linear Combination**. Metode ini digunakan untuk menggabungkan beberapa nilai (score) dengan memberi bobot (weight) pada masing-masing nilai, lalu dijumlahkan. Rumusnya:

$$\text{Score} = (\alpha \times \text{DenseVector}) + ((1 - \alpha) \times \text{SparseVector})$$

Keterangan:

- α (alpha) $\rightarrow$ bobot untuk semantic / vector search (0 – 1)
- DenseVector $\rightarrow$ nilai vektor dari embedding (semantic similarity) yang telah di normalisasi.
- $(1 - \alpha)$ $\rightarrow$ bobot untuk keyword search
- SparseVector $\rightarrow$ Nilai vektor dari pencarian keyword (BM25) yang telah di normalisasi.

Jika nilai α (alpha) diatur menjadi 0.5, maka bobot antara semantik dan keyword adalah sama, sedangkan jika α (alpha) lebih besar dari 0.5, maka bobot semantik lebih besar dibandingkan keyword search.

Hybrid Search

Hybrid Search

Kelebihan

- Akurasi lebih tinggi dibanding satu metode saja
- Tidak Kehilangan Informasi penting
- Paham makna + tetap jaga keyword penting
- Bisa di-tuning (pakai α / bobot)
- Lebih stabil untuk berbagai jenis query

Kekurangan

- Membutuhkan sumber daya komputasi lebih besar dibanding satu metode saja
- Perlu tuning bobot (α) agar optimal
- Butuh normalisasi skor (dense vs BM25 beda skala) jika menginginkan rentang 0-1.
- Lebih lambat dan Integrasi lebih rumit dibanding hanya semantic / keyword saja.



Common Failure Patterns

No	Failure Pattern	Deskripsi	Penyebab	Solusi
1	Missing Relevant Documents	Dokumen relevan tidak diambil (recall rendah)	Embedding kurang baik, chunking salah, top_k kecil	Tingkatkan top_k, gunakan hybrid search, perbaiki chunking
2	Irrelevant Results	Hasil tidak relevan (precision rendah)	Embedding terlalu umum, tidak ada filter	Tambahkan metadata filter, gunakan hybrid search
3	Context Fragmentation	Informasi terpecah di banyak chunk	Chunk terlalu kecil, overlap kurang	Atur chunk size & overlap, gunakan semantic chunking
4	Duplicate / Redundant Results	Banyak hasil mirip / duplikat	Overlap terlalu besar, tidak ada deduplikasi	Kurangi overlap, gunakan re-ranking
5	Poor Ranking	Dokumen paling relevan tidak berada paling atas.	Hanya pakai semantik atau keyword saja.	Gunakan Hybrid Search
6	Too Much Noise	Terlalu banyak context (LLM bingung)	top_k terlalu besar	Kurangi top_k, filtering & ranking

Assignment



Tugas 1

Setelah Anda mempelajari cara untuk membaca dokumen PDF, Chunking, membuat metadata dan melakukan Hybrid Search. Sekarang coba gunakan data atau dokumen PDF [docs_telco_v2.pdf](#), untuk mencari konteks atau teks yang cocok dengan input atau query berikut:

- Berapa jumlah BTS di PT NTD.
- Berikan Langkah registrasi prabayar.
- Layanan apa saja yang ada di PT NTD.
- Berikan daftar paket Internet Paling Murah.

Lakukan pencarian dengan Hybrid Search dan Vector Database yang digunakan adalah **Pinecone** dan **ChromaDB**.

Tugas 2

Anda bekerja di perusahaan Otomotif dan diberikan link (<https://autocatalogarchive.com/mitsubishi/>) yang berisi informasi tentang beberapa mobil mitsubishi dengan berbagai macam bahasa dalam bentuk dokumen PDF. Dari katalog tersebut, gunakanlah semua file PDF yang berbahasa Indonesia atau berkode (ID), misalkan **2018 - Outlander Sport (ID)**.

Buatlah AI yang dapat melakukan Product Search yang dapat menjawab atau mencari konteks yang cocok dengan input berikut:

- Detail spesifikasi Mitsubishi Destinator
- Mobil yang cocok untuk Travel dengan jumlah bangku atau **seating capacity** yang besar.
- Mobil untuk perjalanan jauh yang nyaman
- Mobil Mitsubishi yang irit bahan bakar
- Mobil Mitsubishi hybrid atau electric

Lakukanlah pencarian dengan **Hybrid Search** dan gunakanlah **Pinecone** sebagai vector database.

Batas Pengumpulan: Hari ini, pukul 23.59 WIB
Pengumpulan melalui: Google Classroom



Practice

**Hands on: Chunking, Metadata, Store
embeddings in vector DB and Hybrid Search**

Visit Here



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia